

Nowy czat

Jesteś doświadczonym programistą frontend'u oraz wykładowcą. Wy tłumacz dokładnie co robi każda część poniższego kodu. Odpowiedzi zwięzłe.

```
// src/exercises/1-vanilla-todo.tsx
// Proste Todo List w czystym TypeScript + DOM (bez Reacta)

export function initVanillaTodo() {
  // Pobieramy główny element, do którego wrzucimy nasze Todo
  // const app = document.getElementById('root') as HTMLDivElement;
  const app = document.getElementById('vanilla-root') as HTMLDivElement;
  if (!app) return;

  // Czyścimy zawartość roota (żeby nie nakładało się na Vite template)
  app.innerHTML =
    <div style="max-width: 500px; margin: 40px auto; font-family: Arial, sans-serif;">
      <h2>Vanilla Todo (Moduł 1)</h2>

      <div style="margin-bottom: 20px;">
        <input
          type="text"
          id="todo-input"
          placeholder="Co trzeba zrobić?"
          style="padding: 10px; width: 70%; font-size: 16px;"
        />
        <button
          id="add-btn"
          style="padding: 10px 20px; font-size: 16px;"
        >
          Dodaj
        </button>
      </div>

      <ul id="todo-list" style="list-style: none; padding: 0;"></ul>

      <div style="margin-top: 20px; font-size: 14px; color: #666;">
        <span id="count">0</span> zadań
      </div>
    </div>
  ;

  // Pobieramy elementy z HTML-a
  const input = document.getElementById('todo-input') as HTMLInputElement;
  const addBtn = document.getElementById('add-btn') as HTMLButtonElement;
  const todoList = document.getElementById('todo-list') as HTMLUListElement;
  const countSpan = document.getElementById('count') as HTMLSpanElement;

  // Ładujemy zadania z localStorage
  let todos: string[] = JSON.parse(localStorage.getItem('todos') || '[]');
```

```
// Funkcja renderująca listę zadań
function renderTodos() {
  todoList.innerHTML = "";

  todos.forEach((todo, index) => {
    const li = document.createElement('li');
    li.style.cssText = 'padding: 12px; background: #f9f9f9; margin: 6px 0; border-radius: 6px; display: flex; justify-content: space-between; align-items: center;';

    li.innerHTML =
      <span>${todo}</span>
      <button data-index="${index}" style="background: #ff4d4d; color: white; border: none; padding: 6px 12px; border-radius: 4px; cursor: pointer;">
        Usuń
      </button>
    ;

    todoList.appendChild(li);
  });

  countSpan.textContent = todos.length.toString();
}

// Dodawanie nowego zadania
function addTodo() {
  const text = input.value.trim();
  if (text === "") return;

  todos.push(text);
  localStorage.setItem('todos', JSON.stringify(todos));

  input.value = ""; // czyścimy input
  renderTodos();   // odświeżamy listę
}

// Usuwanie zadania
function deleteTodo(index: number) {
  todos.splice(index, 1);
  localStorage.setItem('todos', JSON.stringify(todos));
  renderTodos();
}

// Obsługa kliknięcia przycisku "Dodaj"
addBtn.addEventListener('click', addTodo);

// Obsługa Enter w polu input
input.addEventListener('keypress', (e) => {
  if (e.key === 'Enter') addTodo();
});
```

```
// Obsługa kliknięcia w przycisk "Usuń" (delegacja zdarzeń)
todoList.addEventListener('click', (e) => {
  const target = e.target as HTMLButtonElement;
  if (target.tagName === 'BUTTON') {
    const index = parseInt(target.dataset.index || '0');
    deleteTodo(index);
  }
});

// Pierwsze wyświetlenie listy
renderTodos();
}
```

Poniżej masz zwięzłe, ale dokładne wyjaśnienie każdej części kodu — linia po linii i blok po bloku.

♦ Eksport funkcji

TypeScript

```
export function initVanillaTodo() {
```

- Eksportowana funkcja inicjalizująca aplikację.
- Można ją wywołać np. w `main.ts`.

♦ Pobranie root elementu

TypeScript

```
const app = document.getElementById('vanilla-root') as HTMLDivElement;
if (!app) return;
```

- Szuka elementu w DOM o `id="vanilla-root"`.
- `as HTMLDivElement` → rzutowanie typu (TypeScript).
- Jeśli element nie istnieje → przerywa działanie.

♦ Wstrzyknięcie HTML (UI)

TypeScript

```
app.innerHTML = `...`;
```

- Nadpisuje zawartość `#vanilla-root`.
- Tworzy cały interfejs:
 - input (pole tekstowe)
 - button "Dodaj"
 - lista `<ul>`
 - licznik zadań

👉 To jest **render statycznego layoutu** (bez frameworka).

♦ Pobranie referencji do elementów

TypeScript

```
const input = document.getElementById('todo-input') as HTMLInputElement;
const addBtn = document.getElementById('add-btn') as HTMLButtonElement;
const todoList = document.getElementById('todo-list') as HTMLUListElement;
const countSpan = document.getElementById('count') as HTMLSpanElement;
```

- Łapie elementy z DOM, żeby nimi sterować.
- Każdy rzutowany na odpowiedni typ TS.

♦ Stan aplikacji (todos)

TypeScript

```
let todos: string[] = JSON.parse(localStorage.getItem('todos') || '[]');
```

- Pobiera dane z `localStorage`.
- Jeśli brak → używa pustej tablicy.
- `todos` = tablica stringów (treść zadań).

♦ Renderowanie listy

TypeScript

```
function renderTodos() {
```

Czyszczenie listy

TypeScript

```
todoList.innerHTML = '';
```

- Usuwa wszystkie `<li>` przed ponownym renderem.

Iteracja po zadaniach

TypeScript

```
todos.forEach((todo, index) => {
```

- Przechodzi po tablicy.

Tworzenie elementu `<li>`

TypeScript

```
const li = document.createElement('li');
```

Stylowanie

TypeScript

```
li.style.cssText = '...';
```

- Inline CSS (szybkie, ale mało skalowalne).

Wypełnienie HTML

```
TypeScript
li.innerHTML = `
  <span>${todo}</span>
  <button data-index="${index}">
```

- Wyświetla tekst zadania.
- Przycisk "Usuń" ma `data-index` → identyfikator elementu.

Dodanie do DOM

```
TypeScript
todoList.appendChild(li);
```

Aktualizacja licznika

```
TypeScript
countSpan.textContent = todos.length.toString();
```

♦ Dodawanie zadania

```
TypeScript
function addTodo() {
```

Pobranie i walidacja

```
TypeScript
const text = input.value.trim();
if (text === '') return;
```

- Usuwa spacje.
- Blokuje puste wpisy.

Aktualizacja stanu

```
TypeScript
todos.push(text);
```

Zapis do localStorage

```
TypeScript
localStorage.setItem('todos', JSON.stringify(todos));
```

Reset inputa + render

```
TypeScript
input.value = '';
renderTodos();
```

♦ Usuwanie zadania

TypeScript

```
function deleteTodo(index: number) {
```

Usunięcie z tablicy

TypeScript

```
todos.splice(index, 1);
```

Aktualizacja storage + UI

TypeScript

```
localStorage.setItem('todos', JSON.stringify(todos));
renderTodos();
```

♦ Event: klik "Dodaj"

TypeScript

```
addBtn.addEventListener('click', addTodo);
```

- Po kliknięciu wywołuje addTodo.

♦ Event: Enter w input

TypeScript

```
input.addEventListener('keypress', (e) => {
  if (e.key === 'Enter') addTodo();
});
```

- Umożliwia dodanie zadania klawiszem Enter.

♦ Delegacja zdarzeń (usuwanie)

TypeScript

```
todoList.addEventListener('click', (e) => {
```

Sprawdzenie targetu

TypeScript

```
const target = e.target as HTMLButtonElement;
if (target.tagName === 'BUTTON') {
```

Pobranie indexu

TypeScript

```
const index = parseInt(target.dataset.index || '0');
```

- data-index → zamiana na number.

Usunięcie

TypeScript

```
deleteTodo(index);
```

👉 Delegacja = jeden listener zamiast wielu.

♦ Pierwszy render

TypeScript
renderTodos();

- Wyświetla dane przy starcie (z localStorage).

Kluczowe koncepcje

- Stan aplikacji → todos
- Render → pełne odświeżenie DOM
- localStorage → persystencja
- Event delegation
- Manualny DOM (bez Reacta)

Jeśli chcesz, mogę pokazać:

- jak to przepisać na React (dla porównania),
- albo jak dodać edycję / checkboxy / filtrowanie.

Jak zamienić wszystkie style="..." na obiektowy zapis w li.style.cssText w poniższym kodzie?
app.innerHTML =

```
<div style="max-width: 500px; margin: 40px auto; font-family: Arial, sans-serif;">
  <h2>Vanilla Todo (Moduł 1)</h2>
```

```
  <div style="margin-bottom: 20px;">
    <input
      type="text"
      id="todo-input"
      placeholder="Co trzeba zrobić?"
      style="padding: 10px; width: 70%; font-size: 16px;"
    />
    <button
      id="add-btn"
      style="padding: 10px 20px; font-size: 16px;"
    >
      Dodaj
    </button>
  </div>
```

```
<ul id="todo-list" style="list-style: none; padding: 0;"></ul>
```

```
<div style="margin-top: 20px; font-size: 14px; color: #666;">
  <span id="count">0</span> zadań
```

```

</div>
</div>
;

```

Nie da się tego zrobić **bezpośrednio w innerHTML** — to tylko string HTML.

`li.style.cssText` (albo styl obiektowy) działa **dopiero na elementach DOM**, nie w stringu.

👉 Czyli: najpierw tworzysz elementy, potem ustawiasz style.

✓ Podejście poprawne (obiektove style)

Zamiast `innerHTML`, budujesz DOM ręcznie:

```

TypeScript
const container = document.createElement('div');

Object.assign(container.style, {
  maxWidth: '500px',
  margin: '40px auto',
  fontFamily: 'Arial, sans-serif',
});

```

♦ Przykład dla całego layoutu

Kontener

```

TypeScript
const container = document.createElement('div');
Object.assign(container.style, {
  maxWidth: '500px',
  margin: '40px auto',
  fontFamily: 'Arial, sans-serif',
});

```

Nagłówek

```

TypeScript
const title = document.createElement('h2');
title.textContent = 'Vanilla Todo (Moduł 1)';

```

Wrapper inputa

```

TypeScript
const inputWrapper = document.createElement('div');
Object.assign(inputWrapper.style, {
  marginBottom: '20px',
});

```

Input

```

TypeScript
const input = document.createElement('input');
input.type = 'text';
input.id = 'todo-input';

```



```
input.placeholder = 'Co trzeba zrobić?';

Object.assign(input.style, {
  padding: '10px',
  width: '70%',
  fontSize: '16px',
});
```

Button

```
TypeScript
const button = document.createElement('button');
button.id = 'add-btn';
button.textContent = 'Dodaj';

Object.assign(button.style, {
  padding: '10px 20px',
  fontSize: '16px',
});
```

Lista

```
TypeScript
const ul = document.createElement('ul');
ul.id = 'todo-list';

Object.assign(ul.style, {
  listStyle: 'none',
  padding: '0',
});
```

Stopka

```
TypeScript
const footer = document.createElement('div');
Object.assign(footer.style, {
  marginTop: '20px',
  fontSize: '14px',
  color: '#666',
});
```

Licznik

```
TypeScript
const count = document.createElement('span');
count.id = 'count';
count.textContent = '0';
```

Składanie całości

```
TypeScript
inputWrapper.append(input, button);
footer.append(count, document.createTextNode(' zadań'));

container.append(title, inputWrapper, ul, footer);
app.appendChild(container);
```

Alternatywa (jeśli chcesz zostać przy innerHTML)

Możesz:

1. zostawić `innerHTML`
2. potem nadpisać style JS-em:

TypeScript

```
const container = app.firstElementChild as HTMLDivElement;
```

```
Object.assign(container.style, {
  maxWidth: '500px',
  margin: '40px auto',
});
```

⚠ Ważne różnice

Podejście	Plusy	Minusy
<code>innerHTML + style=""</code>	szybkie	brak kontroli
<code>cssText</code>	łatwe kopiowanie CSS	string, mniej TS-friendly
<code>Object.assign(style)</code>	czytelne, typowane	więcej kodu

💡 TL;DR

- ❌ Nie przeniesiesz `style=""` do `cssText` w HTML stringu
- ✅ Musisz użyć:

TypeScript

```
element.style.property = value
```

lub:

TypeScript

```
Object.assign(element.style, {...})
```

Jeśli chcesz, mogę pokazać wersję **kompaktową (helper do styli)** albo jak to zrobić bardziej „reactowo” bez Reacta.